RFC 1: How microstructure gets into MOOSE, and how trajectories get out

- **Author:** Tiago Brockert (simulation team)
- **Status:** Draft, open for comment
- **Date:** October 25, 2026
- **Reviewers:** John Scumniotales, Gunnar Larsen, Ellie Ding; Dr. Tianyi Chen (physics sign-off); Dr. Chanho Kim (model side)
- **Location:** `docs/rfcs/0001-geometry-into-moose.md`

1. Problem and context

Our project is to train a neural particle simulation model whose goal is to predict how a porous sample of material (first will be graphite, then we will switch to a Silicon Carbide composite material) will deform and crack under continuous loading. Our training data for this project will come from phase field fracture simulations, following the method done in a paper by Chakraborty, Sabharwall and Carroll (2016) who modeled nuclear graphite in MOOSE with randomly distributed pores inside of a 1mm x 1mm 2D volume.

During our meeting on October 20, Dr. Chen and Dr. Kim helped settle that our fracture physics will run in the MOOSE framework and that John's SEM-to-mesh tool will be placed at the front end of our architecture to aid the geometry. Later, when we have the SiC/SiC composite material dataset, researchers at INL have created an extremely empirical model in BISON that we will hope to replicate and try our model/pipeline at a larger scale with more complex nuclear material. The solver is no longer the big decision to make for our project, which I was originally going to write this RFC about, however, now we need to figure out what must go into and come out of this solver in order to be properly delivered to the model architecture side of the project.

First of all, we must figure out how a microstructure can enter our MOOSE simulator, because Chakraborty represented pores as softer regions inside a uniform mesh (Chakraborty et al. 2016), but John's mesher combines this with the genuine geometry of the image, which are slightly different models.

Then, in order to figure out what we must pass along into the input for the PointConv model, we must export a mesh representation of some sort which decides the locations of mesh nodes are in the training files, then how many of them there are for a general density and an overall mask of what points exist in pores.

So, this RFC is specifically covering which geometry representation that the simulation team should use for the production runs and at which points should the trajectories be uploaded for the model. This is a good time to determine this, because our Repo Checkpoint on Nov 1 needs sample data in our format, our skeleton on week 10 needs the path working from end to end. This week, I'm also doing the initial MOOSE tests on the HPC, so the first

input files that we use for this will allow us to have a pattern that we come back to for future runs.

Right now, this decision is easy for us because we currently havent used many GPU/HPC hours on production or simulation runs yet. Once we start running the MOOSE framework, however, this will take a lot longer. Chakraborty reports 5 to 7 hours on 96 cores (Chakraborty et al. 2016), and so a dataset of some hundred graphite simulations may take tens of thousands of hours split among many GPUs in order to create. Once the dataset exists, each result is tied to a point layout defined by a mesh which overlays our simulations. We will eventually have to revalidate all of these point layouts too, somehow, and once again switch once we have the new SiC-SiC dataset from Dr. Chen. If we do nothing, we will continue producing data via polymer meshes using John's FEniCSx mesher and solver, which relies on a public polymer image dataset (Skärberg et al. 2020) and placeholder material values. This could potentially be validated for polymer separation, but we wouldn't be able to use it to simulate material defects in Dr. Chen's ET-10 graphite, so we would have to scrap all the data later.

2. Options

Option 0: Do nothing.

If we did this, we would continue to generate trajectories from the FEniCSx end to end prototype and would be committing to a different material for simulation purposes. We would thus have differences in the data representation, action + load conditioning (the current solver applies horizontal tension only). This wouldn't cost too much upfront, but the model would be unrealistic for simulating graphite physics and would have to be redone for any more specific graphite or SiC-SiC simulations/datasets. This would also make it much more difficult to validate the phase field fracture results, and we would be working in a framework that Dr. Chen isn't as familiar with.

Option A: We make conformal meshes with John's mesher and export them at the mesh nodes.

Our pores will become holes, the pore generator + mesher will be used to write a Gmsh .msh file that MOOSE will read, solve, and an exporter will write the displacement of every node and the damage at our different load steps. The slightly more difficult part about this option is that we must make sure that our physical holes in the mesher and the soft regions in the option show the same fracture behavior, but we would completely own the code for the mesher and exporter.

Option B: MOOSE-native geometry method used in the Chakraborty paper

For this method, we start from a completely uniform mesh and assign pores as regions without stiffness via lists of pores that were generated with Gunnar's generation code. We would export all of these nodes and just make sure we're masking out the ones which currently lie inside of pores. The trouble here is that many of the nodes will be sitting inside the pores entirely and therefore won't carry important info for the simulation. Also, I'm not

sure how we would work the mesher in this case. Perhaps it would still be very useful when receiving the new SiC-SiC dataset for which, because our physics model is much more empirical and may not be *regular* enough to provide enough neural training data, we could use the mesher on the new, larger and more complex composite fracture data to create a list of important points around our voids and other critical regions in order to create pointclouds of important nodes for tracking via AllTracker, which could then be used for fine tuning and improving our empirically derived model with real, experimentally derived data.

Option C: Export the solver physics onto a regular, square query grid.

For this method, we would use option A or B where we would either use the conformal meshes or just use the MOOSE-native geometry and solve them in MOOSE normally with the phase-field fracture method, but then we would put the results back onto a regular grid of points and save those instead. This may spread out some of the damage band near the cracks and we would need to perform some confidence check or validation to make sure that this interpolation is a valid move, and how much this compute/confidence tradeoff will affect the accuracy of our results.

3. Analysis

Concern	0: Do nothing	A: Conformal + nodes	B: Soft regions	C: Query grid
Physics match to Chakraborty	Poor (wrong material, wrong loading)	Close, needs an equivalence test	Exact representation	Inherits A or B
Stress at pore edges	n/a	Exact boundary	Depends on refinement near the edge	Smoothed
Points per sample	n/a	Only solid material	Includes pore interiors (masked)	Fixed count
Fixed node set per run	Yes	Yes (no mid-run remeshing)	Only if refinement is done at setup	Yes by construction
Matches real data layout	n/a	Like tracker points on the surface	Mostly	Like DIC, which is already a grid
Code we maintain	FEniCSx solver	Mesher + exporter	MOOSE inputs only	An interpolation step
Reuse for BISON later	No	Yes, same mesh files	Yes	Yes

Physics Match: Option B is literally what Chakraborty ran, which makes his published numbers (strength around 10.5 MPa for his first volume, the stepwise load drops) directly comparable. Option A changes one modeling choice between our approach and Chakraborty's setup. A soft pore still carries a small residual stiffness which allows the crack field to spread into it (Chakraborty et al. 2016; MOOSE ComputeCrackedStress documentation). Our pores for option A wouldn't be there. I think that the two should agree

on the peak load and where the cracks start because that the pores are where the stress initiates to begin with and we would still be modelling the outside of these pores.

Cost and performance: We will start by simulating a 4 um grid over 1mm^2, which is 62500 elements before downsampling (if we choose to do this). About 20% of these points would live inside of pores. Option A should cut down a little compute because these 20% of pore points wouldn't even be in the mesh. John's mesher builds a mesh in 9.3 seconds on average, so this shouldn't present a bottle neck. The finite element step, however, may cost a lot of compute depending on how many unknowns MOOSE has to solve for and how many load steps our fideo data is.

I slightly prefer option A for the fact that our draft data format stores displacement at a fixed set of points over time, plus a mask that ensures that the points are valid. In B, the nodes inside of pores have to be constantly masked for every file, which we don't currently have a solution for, and it would also break our adaptive refinement runs if our node set changes mid-trajectory. We also must ensure that we can change the size of our mesh if needed, because we were given the requirement that element size must be at or below half of the length scale l or else damage bands may be unresolved which will overestimate the material toughness (Bourdin et al. 2008) .

Also, Dr. Chen's Box folder already seemed to have DIC exports with one CSV per frame with positions and displacements on a regular grid (Chen, "Kairos Data" Box folder, 2026) . This means we could pretty easily try option C as an additional exporter. This also allows us to compare both our simulated and real data, which may be helpful for our uncertainty calculations later when we train another model to predict the sources and magnitudes of error in our predicted simulations. I don't want to use this as our main exporter, however, because having very rigid and thin fractures is optimal for training (if the damage band is too spread out, it gives the model too much variation).

Maintainability and security: Option B keeps everything inside of MOOSE input files. Option A just adds to John's current mesher. This has an API currently in progress. None of these seem to raise a security concern, however, I will discuss confidentiality soon.

Recommendation: Option A for production runs, with these conditions:

1. Geometry enters MOOSE as conformal Gmsh meshes from John's mesher (image-derived) or Gunnar's generator (synthetic), with tagged boundaries. We must describe a training data split for these two sources of data.
2. No adaptive remeshing during a run. Refinement happens only in the mesh file.
3. The primary export is node displacement and damage at fixed load steps, in our data format.
4. A query-grid exporter (Option C) is perhaps built later as a secondary path, for comparing against DIC.
5. Every geometry is stored as its *recipe* (generator seed and parameters, or image ID, crop and pixel size), not only as a mesh file, so it can always be regenerated.

How we will know it was the right call. Each check has a pass/fail threshold, and if check 3 fails, we switch to Option B before any production runs.

#	Check	Pass if	Owner
1	Import: a small mesh from John's tool loads into MOOSE	Boundary tags 1–5 present, node count matches the mesher's report	Tiago + John
2	Benchmark: Chakraborty's single-edge-notched specimen (no pores) in MOOSE	Peak stress within 10% of his reported curve	Tiago
3	Equivalence: one circular pore in a plate under tension, run as a hole (A) and as a soft region (B)	Peak load within 5%, crack starts at the same pore edge	Tiago
4	Cost: same volume in A and B	Record unknowns and wall time; A should not be slower	Gunnar
5	Mesh convergence at h, h/2, h/4 ($h \leq l/2$)	Peak load changes under 5% between the two finest meshes	John
6	Export round trip: MOOSE output → data file → loader	Displacements match to float32 precision; N fixed across steps	Tiago + Ellie
7	Reproduction: a Chakraborty-style volume with his Table 2 values	Strength within 10% of about 10.5 MPa	Tiago, reviewed by Dr. Chen

Checks 1 and 6 are small enough to run in CI on a toy mesh demo, which also covers part of the Repo Checkpoint 1 requirement.

Reversibility. Until check 3 passes and production starts, reversing is easy. After that, the cost of switching is regenerating the data. In order to allow for reversability, we must make sure that we're saving our geometry information, and that we keep the exporter independent from the solver. If we do this, we can switch to choice B without having to rebuild the whole pipeline.

4. Constraints

- **Licensing.** MOOSE is LGPL-2.1 (stated in its source headers). Gmsh is GPL; we use it as a tool and our repository is already public, so that is compatible. BISON is not

open source; access goes through INL. Our pipeline must therefore run without BISON, and anything BISON-specific stays out of the public repo.

- **Data use.** Dr. Chen approved putting his ET-10 graphite data in our public repository. The public FIB-SEM polymer images John used are CC BY 4.0 and need attribution if derived meshes are published (Skärberg et al. 2020). The license on John's newer 300-mesh set from another paper is unknown, so it stays out of the repo until checked.
- **Confidentiality boundary.** The future SiC/SiC data is confidential, and a mesh built from a confidential image *is* that microstructure. Meshes, recipes and trajectories derived from SiC images must follow the same rule as the images: private storage only, never the public repo or cloud AI tools. The `source` field in each data file lets the repo's CI guard catch this automatically.
- **IP.** Chakraborty's model is published, so reimplementing it is fine with citation. No INL input files are involved, because none were available to us now.

5. Delegation and validation

I used Claude for helping compare some of John’s code against some of the continuum mechanics derived formulas in Chakraborty’s paper. I also used it to help look into the phase field objects and the testing suite that we may be able to utilize on MOOSE to validate our fracture approach. Lastly, I used it to help structure (not to write!) this RFC, and research and help word our constraints and potential licensing issues. For everything I mentioned, I made sure to check myself for validity and ask questions in order to understand confusing topics. I also have brought up questions I still have after its explanations to the professors.

References

B. Bourdin, G. A. Francfort, J.-J. Marigo, "The variational approach to fracture," Journal of Elasticity 91 (2008): 5–148.

P. Chakraborty, P. Sabharwall, M. C. Carroll, "A phase-field approach to model multi-axial and microstructure dependent fracture in nuclear grade graphite," Journal of Nuclear Materials 475 (2016). https://doi.org/10.1016/j.jnucmat.2016.04.006

T. Chen, "Kairos Data" (ET-10 graphite test data: in-situ SEM videos, DIC exports, mechanical data, XCT), Oregon State University Box folder, shared with the project team, 2026.

MOOSE Framework, "ComputeCrackedStress," Idaho National Laboratory documentation. https://mooseframework.inl.gov/source/materials/ComputeCrackedStress.html

T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P. Battaglia, "Learning Mesh-Based Simulation with Graph Networks," International Conference on Learning Representations (ICLR), 2021.

A. Skärberg et al., "Convolutional neural networks for segmentation of FIB-SEM nanotomography data from porous polymer films for controlled drug release," dataset, Zenodo, 2020. https://doi.org/10.5281/zenodo.4317170